



Blunder prediction in chess

Yarden Rokach¹ · Bracha Shapira¹

Received: 4 July 2025 / Accepted: 27 January 2026 / Published online: 16 February 2026
© The Author(s) 2026

Abstract

The ability to predict blunders in chess plays a crucial role in improving players' performance and enabling strategic decision-making. We introduce a novel, scalable, and personalized blunder prediction model for chess. Unlike prior work requiring a separate model per player, our unified architecture learns a collaborative user embedding space, allowing it to generalize weaknesses across players and new users. Our hybrid model, inspired by Deep Factorization Machines (DeepFM), fuses a frozen pre-trained CNN (for board embeddings) with dynamically learned user embeddings to model player-board interactions while still utilizing metadata about the state of the board and the user. We demonstrate that this latent 'blunder profile' is a significantly more powerful predictor of error than a player's explicit Elo rating. The system achieves state-of-the-art performance (0.801 AUC) on both immediate and non-immediate blunders, offering an efficient and data-sparse-friendly solution for personalized chess analysis. Ultimately, this approach demonstrates the practical viability of deep personalization in complex strategy games, facilitating highly efficient, user-centric learning environments.

1 Introduction

Chess, an age-old contest of intellect and strategy, has long been a captivating subject of study and reflection across cultures and centuries. Its intricate web of tactics and positional dynamics has lured scholars, enthusiasts, and thinkers into a realm where foresight and calculation merge in a symphony of complexity. Yet, beyond its traditional allure, chess has served as a fertile ground for scientific exploration, particularly within the realms of computer science and machine learning.

In chess, a player's or machine's skill level is measured by their Elo rating, a numerical score that adjusts after each match based on the outcome and the opponent's rating. The rating increases with wins and decreases with losses, reflecting the relative strength of the players [1].

Algorithms for chess playing have continued to improve and recently reached a much higher Elo ranking than their human counterparts, especially thanks to advancements in deep reinforcement learning (DRL) algorithms such as

AlphaGo and its successor AlphaZero [17]. More recently researchers have started to examine the alignment of AI-based models with human player behavior, showing that moves performed by machines can be hard to understand by humans and that using DRL can reduce the ability to model human players [11]. These recent attempts have three main motivations: First, they use chess as a testbed to demonstrate modeling granular human behavior where AI already exceeds human ability. Second, they use chess model to better understand human skills and their decision-making process. Third, it enables tighter human-AI interaction and the development of personalized chess-learning tools that aim to help human improve their chess skills.

McIlroy-Young et al. [12] introduce a method for learning personalized models of individual behavior in chess. The authors build on prior work [12] using deep learning to predict human chess moves at a coarse skill level. They use their previous an open-source AlphaZero-like chess engine called Maia [11] and apply transfer learning techniques to adapt it to individual players. By fine-tuning Maia's neural network on a player's past games, they are able to significantly improve prediction accuracy compared to population-level models. On average, their personalized models achieve 4-5% higher move prediction accuracy than the best baseline model per player. The personalized models even outperform on predicting blunders, suggesting they learn a player's specific weaknesses. A key result is that

✉ Bracha Shapira
bshapira@bgu.ac.il
Yarden Rokach
rokachj@post.bgu.ac.il

¹ Ben-Gurion University of the Negev, Beersheba, Israel

the personalized models capture enough individual aspects of a player's style to perform near-perfect "stylometry" - identifying a player from their moves alone. By testing each model on games from other players, they can correctly identify the player who played a sample of games 98% of the time out of 400 players. The models can even identify players' blunder styles, which indicates that their representation captures a player's unique strategic decision-making. Overall, the work shows the value of learning human policies at an individualized level.

Blunder prediction remains a domain with relatively limited research exploration, exemplified by prior endeavors like that of [11]. This study embarked on predicting the likelihood of a human player committing a blunder in a specific board context using their Elo rating without any personalization. Notably, each chessboard position in a game boasts an annotated score attributed by the Stockfish chess engine, subsequently correlated to winning probabilities. Within this framework, a move is identified as a blunder if the winning probability undergoes a reduction of at least 10%. Whereas [12] does model individual players and their blunder style but does not dive into blunder prediction. Blunder prediction is motivated by the need to model human error—not just optimal play—so that AI can teach better, play more realistically, and support decision-making under real-world cognitive constraints.

We differentiate and predict two types of blunders : immediate blunders and non-immediate blunders [15]. Where the consequences of an immediate blunder become clear immediately, while for non immediate blunders the consequences appear obvious in later moves. Formally, an immediate blunder occurs when, following the blunder move, the opponent responds with the best move, the game either ends in a checkmate or the player who made the blunder also plays the best move in response and still experiences a loss of material compared to the position before the blunder. While a non-immediate blunder results in material loss or the loss of the game when the opponent plays their second move after the blunder or onwards. Typically immediate (tactics) blunders are performed by less skilled players or as a result from a momentary lapse in judgment or loss of focus, while non-immediate (strategic) blunders arising due to a lack of understanding of the board and its strategic complexities and can be performed also by skilled players [15]. In addition, to predicting if the user's next move is a blunder we forecast whether a user is prone to a specific type of blunder. For instance, this information could be utilized to tailor puzzles for users, focusing on tactical immediate blunders for novice users and provide scenarios where players are more likely to make strategic blunders for more skilled users. We show that we obtain similar performance for both blunder types.

While the blunder prediction approach presented [11] solely relies on user metadata to forecast whether the subsequent move is a blunder or not, our model integrates user embeddings describing the user's behavior and allows incorporating patterns learned about similar players to facilitate personalized predictions resulting in a more accurate and more efficient model than the state-of-the-art. Moreover, instead of integrating the raw board state into our model, we incorporate a pre-trained model trained to play chess using human chess data to serve as a board embedding [4]. This approach leverages the assumption that the pre-trained model captures human gameplay patterns and strategies, providing a more nuanced and effective representation. Specifically, to encode the board state, we employ the RISEv2-mobile architecture from the CrazyAra engine [3] as a frozen feature extractor. Unlike traditional convolutional networks that utilize aggressive down-sampling, this architecture is uniquely improved for board games: it avoids pooling operations to strictly preserve spatial dimensions.

The contributions of the proposed method are the following:

- Our work breaks new ground by shifting from the 'one-model-per-player' paradigm [McIlroy-Young 2022] to a single, collaborative model. This is not a trivial distinction; the per-player approach is computationally expensive and struggles with new users or those with few games. Our unified framework solves this by creating a shared embedding space, marking the first application of modern collaborative filtering techniques to personalized chess behavior modeling.
- The proposed model lets players with similar embedding to generalize from one another. Unlike previous approaches [12] used to model chess players which required a dedicated unique model for every player, this approach reduces training time, improves efficiency, and requires less data per user.
- We also propose prediction of the specific type of blunder a user is likely to make (immediate or non-immediate), which can be useful for personalizing tutoring systems.
- We have examined the model's performance across different phases of the game and for users with varying skill levels, providing relevant insights on the results.

2 Related work

Our work is inspired by methods that were mainly developed in the user modeling and recommender systems domain. User modeling is the process of building up and modifying a conceptual understanding of the user based on specific

background knowledge and objectives [5], including needs, preferences, behavioral patterns, skills, cognitive processes and specific components, in order to adapt systems to the user's specific needs and predict future actions [5, 20].

User modeling has been used in the chess domain for modeling and predicting the moves of both a specific user and the most common move of a group of users with similar playing level [11, 12]. Previous work to model and personalize users in chess used a separate model per user and transfer learning from a baseline model [12]. In our work, we would like to model users by using a single collaborative model for all users. Doing that will allow us to share behavior patterns between users with similar characteristics that led to better predictive performance, particularly when the available training data per user is sparse. This also improves the simplicity of the training process and its efficiency.

User embedding is a technique that can be used for static or dynamic [7, 8] user modeling. It involves mapping a user to a vector of numbers, which serves as the user's unique identifier [13, 21]. User embedding has multiple usages, e.g., it can be used in recommendation systems to predict ratings of the user to items, in social media and forums to suggest personalized posts, or to predict the next action of the user [2, 13, 18, 21, 23]. Embeddings of recent interactions can be dynamically integrated into the accumulated user history [9]. This approach has been used in tasks such as personalized text classification [9], predicting event streams [19], recommendation systems [22] and building K-Nearest Neighbor (KNN) models [16]. In this study we will use multiple user embeddings for each activation of the model in order to make it personalized. Users will be mapped to their representative vectors which are updated during training using gradient descent.

User modeling has been used in the chess domain for modeling and predicting the moves of both a specific user and the most common move of a group of users with similar playing level [11, 12]. Previous work to model and personalize users in chess used a separate model per user and transfer learning from a baseline model [12]. In our work, we would like to model users by using a single collaborative model for all users. Doing that will allow us to share behavior patterns between users with similar characteristics that led to better predictive performance, particularly when the available training data per user is sparse. This also improves the simplicity of the training process and its efficiency.

A paramount advantage of our collaborative architecture is its ability to facilitate continuous cross-user knowledge transfer. When new data is introduced from any player—even a completely new one—the training process updates the shared weights of the Deep and Factorization components. This results in an improved, generalized feature mapping for the entire population, older players included. While

the same would require re-training all the old models when using the transfer learning method. Consequently, the predictions for existing (old) users are immediately enhanced because their personalized embeddings are now being fed into a more accurate and robust shared knowledge base. This dynamic improvement, where the model for one user benefits from the new experiences of all other users, creates a powerful, self-improving system that significantly surpasses the data efficiency and adaptability of static, per-user and transfer learning methods.

Unlike transfer learning, which frequently demands modifying a large number of architectural weights when adapting to new users, our collaborative model achieves rapid personalization. By efficiently initializing the new user's embedding, and limiting the learning process to optimizing only the embedding vector, drastically reducing the number of trainable parameters and consequently lowering the data prerequisites for achieving accurate cold-start predictions.

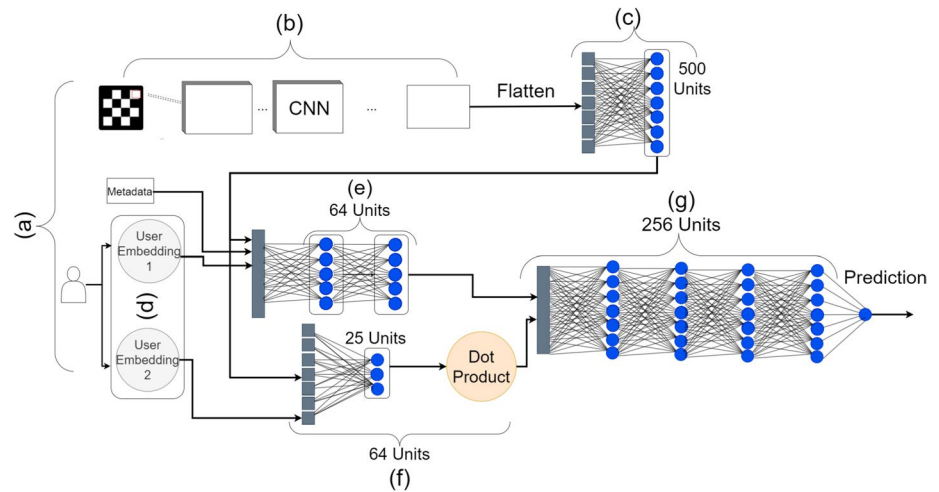
In the context of recommender systems, deep learning models have demonstrated the ability to learn intricate user-item interactions from raw data, thereby alleviating the need for extensive feature engineering [24]. In our context, the chess board can represent an item with which the user interacts.

Factorization Machines (FM) models are a widely recognized class of models that excel in capturing second-order feature interactions [14]. They are particularly adept at dealing with sparse datasets and can efficiently model feature interactions without becoming computationally prohibitive. FM models linearly combine the features and their pairwise interactions through factorized parameters, allowing them to capture complex relationships among features effectively. DeepFM is a pioneering model that blends the strengths of deep learning and factorization machines to achieve impressive results in recommendation systems and click-through rate prediction tasks [6]. DeepFM represents a significant advancement in the realm of collaborative filtering-based recommendation systems. In this paper, we demonstrate that a model inspired by DeepFM architecture can be used for learning users profiles in the chess context.

3 Methodology

3.1 Blunder prediction

Figure 1 displays the full pipeline for blunder prediction. The method involves the prediction of a user's likelihood to make a blunder in their subsequent move, given a specific board and supplementary metadata. To achieve this objective, we adopt a Deep Neural Network architecture inspired by Deep Factorization Machine that includes both, a deep

Fig. 1 Full pipeline of the blunder prediction method

component and a factorization component. The final architecture incorporates a Dedicated user embedding scheme, featuring two distinct embedding layers to represent the user separately for the Deep and Factorization components. This design choice is empirically justified, having yielded the highest predictive performance metrics, as shown in the architectural comparisons presented in Table 1. Following is a detailed description of the architecture:

(a) The model takes as inputs a chess board, the user's identifier and the move's ply (combined move count of both players). Prior to applying the model, the players rating (if incorporated) and move ply inputs undergo normalization

using standard scaling. (b) The board is passed through a frozen pre-trained CNN model trained to play chess introduced in [4]. The model is a 40-layer deep structure comprising a $3 \times 3 \times 256$ convolutional layer and 13 inverted residual blocks. Each such block consists of a 1×1 convolutional layer, a 3×3 deconvolution layer and a second 1×1 convolutional layer. It follows a 'pyramid' design where feature depth is progressively increased—starting at 128 channels in the first block and expanding by 64 channels per block to reach 896 channels in the final layers. By utilizing these MobileNetV2-inspired blocks alongside Squeeze-and-Excitation (SE) layers, the system allows us to leverage

Table 1 Main characteristics for the different detailed architectures and their AUC and AUC-PR scores for blunder prediction

Arch	Board embedding	User-board join	User embedding	Deep component	Joint head	AUC	AUC-PR
1	Siamese Board Embedding	Inner Product	All Shared	256, 256, 128	256, 128, 32	0.76	0.202
2	Siamese Board Embedding	Concatenation	All Shared	256, 256, 128	256, 128, 32	0.753	0.19
3	Siamese Board Embedding, Classifier Board Embedding	Inner Product	Dedicated	512, 512, 256, 256, 128, 128	512, 256, 128, 32	0.766	0.206
4	Siamese Board Embedding, Classifier Board Embedding	Inner Product, Concatenation	Dedicated	512, 512, 256, 256, 128, 128	512, 256, 128, 32	0.769	0.207
5	Siamese Board Embedding, Raw Board	Inner Product, Concatenation	Dedicated	512, 512, 256, 256, 128, 128	512, 256, 128, 32	0.775	0.213
6	Siamese Board Embedding, Classifier Board Embedding	Inner Product	Factorization Shared	None	512, 256, 128, 32	0.77	0.209
7	Siamese Board Embedding, Classifier Board Embedding	Inner Product, Concatenation	All Shared	512, 512, 256, 256, 128, 128	512, 256, 128, 32	0.765	0.207
8	Siamese Board Embedding, Raw Board	Inner Product, Concatenation	All Shared	512, 512, 256, 256, 128, 128	512, 256, 128, 32	0.758	0.196
9	Classifier Board Embedding	Concatenation	All Shared	256, 256, 128	256, 128, 32	0.73	0.171
10	Raw Board	Concatenation	All Shared	256, 256, 128	256, 128, 32	0.613	0.155
11	Raw Board	Concatenation	All Shared	256, 256, 128	256, 128, 32	0.752	0.183
12	Siamese Board Embedding, Classifier Board Embedding	Inner Product	Shared	512, 512, 256, 256, 128, 128	512, 256, 128, 32	0.757	0.196
13	Siamese Board Embedding, Classifier Board Embedding	Inner Product, Concatenation	Shared	512, 512, 256, 256, 128, 128	512, 256, 128, 32	0.76	0.199
14	Pretrained Model	Inner Product, Concatenation	Dedicated	512, 256, 256, 128	512, 256, 128, 32	0.796	0.228

rich, pre-learned tactical representations and complex motif detection without the computational overhead of training a deep CNN from scratch. The output of the layer prior to the penultimate layer is subsequently used. Initially, (c) the board representation undergoes a fully connected neural layer with 500 units and a ReLU activation function. (d) The user's identifier is embedded into two 25-dimensional vectors. (e) The deep component: Subsequently, the first user embedding, the 500 features representing the board, the user's rating, the opponent's rating and the move's ply are combined into a single vector and fed into another fully connected neural network of two layers containing 64 neurons and ReLU activation function, resulting in a vector denoted as 'x.' (f) The factorization component: Additionally, the board representation is processed through a fully connected neural layer with 25 units and a ReLU activation function. An inner product is then applied to the second user embedding and the board representation, generating a new vector denoted as 'y.' The inclusion of this factorization component is a key innovation, as it explicitly models the nuanced, second-order interactions between a player's latent weaknesses (their embedding) and the board's specific features, which the deep component alone may not capture. (g) The head: The vectors 'x' and 'y' are concatenated and passed through another fully connected neural network of four layers, each containing 256 units and using ReLU activation function. The last layer is composed of 1 unit and uses the Sigmoid activation function in order to output a single value that represents the predicted probability of the player to make a blunder on the next move.

The model architecture was implemented using the TensorFlow framework. We utilized the Adam optimizer for gradient descent, employing a learning rate of 0.0001. To address the class imbalance inherent in the dataset, we employed a weighted binary cross-entropy loss function, assigning a weight of 0.08 to the loss contribution from non-blunder moves.

3.1.1 Hyper parameters

The model encompasses four hyperparameters, pertaining to both, the initial neural network that processes the inputs which are the board, first user embedding, move ply, and player's rating (e), and the subsequent neural network that culminates in the prediction output (g). For each network, there are two hyper parameters, the number of layers and the size of each layer. To determine the optimal values for these hyperparameters, the 'Optuna' framework was employed, utilizing the Tree-structured Parzen estimator (TPE), a Bayesian optimization technique, to maximize the Area Under the Curve (AUC). We have assessed the number of layers and the size of each layer for both neural networks,

with the number of layers between 1 and 5 and the size of each layer as 32, 64, 128, 256 or 512 for a total of 25 different combinations for each network.

Using TPE and an allowance of 10 trials, the best hyperparameters were 1 layer for the initial network and 5 layers for the neural network leading to the prediction, and 32 and 128 units per layer for each neural network, respectively

We acknowledge that limiting the Optuna search to 10 trials utilizing TPE algorithm is relatively few for fully exploring the 5×5 hyperparameter grid space for both the initial and joint neural networks. This limitation was primarily driven by computational constraints and the necessary extensive training time required for each full model evaluation, particularly during the initial architecture search across multiple cohorts. Crucially, the search was confined to optimizing the final two stages of the model (components (e) and (g)), while the highly influential user embeddings dimension (25 units) was determined through prior, separate experimentation. Given that the core architecture was already largely stabilized by the DeepFM and frozen CNN design, the 10-trial optimization proved sufficient to locate a stable and highly performant configuration that maximized the validation set AUC (0.801) within the imposed computational budget. Further marginal gains would likely necessitate a disproportionately large increase in training resources.

3.2 Predicting different blunder types

In addition to predicting the probability of a blunder in the next player's move, we also examined and suggest to predict the type of the predicted blunder, immediate or non-immediate blunder. The goal of this categorization to two blunder types, is to differentiate between immediate blunders commonly resulting from a momentary lapse in judgment or loss of focus from those arising due to a lack of understanding of the board and its strategic complexities. The latter is more indicative of the user's abilities and weaknesses. Moreover, such blunders are typically less straightforward to comprehend, rendering them more valuable for in-depth study.

3.2.1 Prediction

Given the distinctions and consequences associated with each blunder type, our aim is to forecast whether a user is prone to a specific type of blunder. For instance, this information could be utilized to tailor puzzles for users, focusing on scenarios where they are more likely to make strategic blunders.

To anticipate whether a user is inclined to commit a particular type of blunder, we developed two additional

models, each designed for a distinct blunder type. These models shared the architecture outlined in Section 3.1 but diverged in their training approach. For each blunder type, the model was specifically trained to predict the probability of that particular blunder type. This entails assigning a target value of 1 for a move corresponding to the relevant blunder type during training and 0 otherwise, even if the move constituted a blunder of a different type.

4 Dataset and experiment settings

The dataset used for evaluation consists of historical chess game records obtained from Lichess.org, an open-source chess platform. The dataset was preprocessed and is commonly used in numerous prior research [11]. We chose this dataset of online games because it provides a large, easily accessible source of data that reflects the current chess landscape. With the growing popularity of online play, this dataset captures a diverse range of player skill levels, making our model more relevant and applicable to today's chess environment.

We have used a total of over 3.9 million chess moves played by a diverse set of users, including 311,113 blunders. The dataset contains the following information: the id of the user that is to play the move, the arrangement of pieces on the board, metadata which consists of the rating of the user and their opponent, and how many moves have been previously played. Finally the dataset contains data about whether the move was a blunder or not. To ensure a comprehensive evaluation and making sure no data leakage occurs while evaluating the model, the dataset is composed out of two subsets: (subset100) the 100 players who have made the most blunders which will be used for model tuning and validation, and (subset750) 750 users selected based on three factors: their rating, the number of blunders in the dataset, and their blunder-to-non-blunder ratio. The range of each factor was divided into five bins based on Equal-Frequency Binning. Then we designed a full factorial experiment such that for every combination of these three factors (altogether 125 combinations), we randomly selected 6 corresponding players. This results in 750 players. The general ratio for blunders to non-blunders in the dataset was 1 blunder to 12.6 non-blunder moves.

Each of the two subsets (subset100 and subset750) was split into training and evaluation sets based on the chronological order of games for each player. Specifically, the most recent 20% of games for every player were held out as the evaluation set, while the remaining 80% of earlier games were used as the training set. This ensures no data leakage between the training and evaluation sets, as the model is only trained on old games and tested on unseen newer

games for each player. Using a chronological split helps evaluate the model in terms of generality to future games, as opposed to a random split which may allow training and testing on games played close together in time or even on games that the trained games. The model's ability to perform well on future games is a more realistic test of its real-world chess playing skill. It is important to note that a game might appear both in the train set and the test set, whereas the moves of one player will be used in the train set and the moves of the other in the test set.

Subset100 comprises a training set with 1,048,823 moves, which includes 82,424 blunders, and a test set with 262,207 moves, encompassing 20,528 blunders. Additionally, within the first training set, there are 17,659 immediate blunders and 64,765 non-immediate blunders, while the test set contains 4,350 immediate blunders and 16,178 non-immediate blunders. Meanwhile, subset750 encompasses a training set with 1,299,902 moves, housing 100,737 blunders, and a test set with 324,974 moves, featuring 25,000 blunders. In the second training set, there are 21,853 immediate blunders and 78,884 non-immediate blunders, with the test set containing 5,287 immediate blunders and 19,713 non-immediate blunders.

In the initial stages of our research (referred to later as initial experiment), which involved discovering the initial architecture and hyperparameter tuning, we exclusively utilized the subset100 of the dataset. Subsequently, for model evaluation, we employ the training sets of both subsets to train the models, reserving the test set of subset750 for model evaluation purposes.

To determine the optimal training duration and prevent overfitting, we employed an early stopping protocol. Training proceeded until the Area Under the Curve (AUC) metric on the validation set showed no improvement for three consecutive epochs. Following the identification of this optimal epoch count, the final model architecture was then retrained on the entire chronological training dataset to maximize data utilization for deployment.

5 Evaluation

5.1 Evaluation metrics

Due to the imbalanced nature of the dataset, whereas 8% of the moves are tagged as blunders, the performance of the blunder prediction model was assessed using two widely-used evaluation metrics: the AUC and the AUC-PR. The AUC measures the ability of the model to discriminate between blunders and non-blunders across different threshold values, while the AUC-PR evaluates the model's precision-recall trade-off.

5.2 Initial Experiments for optimal blunder prediction architecture selection

During the initial experiment, we evaluated various architectures. For this, we used a cohort of 100 players (subset100) with the most number of blunders for training and evaluation. It should be noted that once we found the best architecture, we use a different cohort of players (subset750) in the final experiments. The following subsections describe each of the examined architectures

We have assessed 14 distinct model architectures, shown in Table 1, which are combinations of variations across five components of the architecture. We highlight the primary variations that exhibited notable differences, while acknowledging that there are additional minor distinctions not detailed. In the following section we outline the key distinctions that characterize various architectural configurations. Five salient differentiators emerge within this framework:

- Board Embedding** - We ventured into several techniques for board embedding. Among these endeavors, we delved into training a Siamese network, tasked with discerning the similarity between boards based on their motifs. Motif is a fundamental concept in chess. Motifs denote recurrent patterns, themes, or tactical strategies frequently encountered during gameplay. Additionally, we explored the application of a convolutional neural network (CNN) to classify boards according to their motifs, followed by a transfer learning approach where we severed the CNN's classification head and repurposed the network solely for the board embedding task. We also experimented using the original board representation, Recognizing that using the pre-trained model as an embedding method leverages an CNN, we aimed to enable the model to glean features directly from the original board configuration. To achieve this, we introduced a CNN architecture preceding the fully connected layer comprising 500 units. This preliminary CNN cascade encompassed five layers, each endowed with a distinct filter count and size – 32 filters with a 4x4 kernel, 64 filters with a 3x3 kernel, 128 filters with a 2x2 kernel, followed by two layers each equipped with 128 and 256 filters and 2x2 kernels. Notably, all layers employed padding to preserve spatial dimensions. To sum up, for the board embeddings in the model, we tested the four options explained above.: embeddings created through the Siamese network, those derived from the classifier auxiliary task, raw (original) board embeddings, and the embeddings extracted using the pretrained model. Evaluating each of these board embedding methods is crucial to understanding their impact on the model's performance, as different embeddings may capture varying
- aspects of the board's strategic complexity and influence the accuracy of blunder predictions.
- User-Board join** We examined the combination of board and user embeddings, primarily through concatenation in the deep component or an inner product within the factorization component. We wanted to determine which combination best enhances the model's ability to integrate user-specific features with the board state, ultimately improving the model's predictive power.
- User Embedding** - The third delineation encapsulates whether a shared embedding is used in both factorization and deep components. When multiple board embeddings are present, a distinct user embedding may engage with the factorization component for inner product computation alongside each board. Three categorizations emerge: 'Shared,' with a single user embedding spanning both components; 'Factorization Shared,' with unique user embeddings for the deep and factorization components, using the same embedding for the inner product when multiple board embeddings are used; and 'Dedicated,' with distinct user embeddings for each component, and individual embeddings for every board input and the deep component. Evaluating these different configurations is vital to understanding how the interaction between user and board embeddings across components impacts the model's overall ability to predict blunders, as it may reveal the most effective way to integrate and leverage user-specific information within the model.
- Deep Component** - Our exploration of the deep component structure encompasses three distinct architectures. The initial architecture comprises three layers with sizes of 256, 256, and 128. The second architecture involves four layers sized at 512, 256, 256, and 128, respectively. Lastly, the third architecture integrates six layers with sizes of 512, 512, 256, 256, 128, and 128. The column value in the table denotes the count of layers employed for the architecture's construction. Evaluating these different deep component structures is essential as it allows us to assess how varying depths and layer configurations impact the model's capacity to effectively learn from and represent the intricate relationships between board positions and user tendencies, ultimately leading to more accurate blunder predictions.
- Joint Head** - Our exploration of the head component structure encompasses two distinct architectures. The initial architecture comprises three layers with sizes of 256, 128, and 32. The second architecture involves four layers sized at 512, 256, 128, and 32, respectively. The column value in the table denotes the count of layers employed for the architecture's construction. Similarly to the deep component we asses how varying depths and layer sizes in the final head architecture affect the model's blunder prediction capability.

Out of all the Architecture evaluated in the initial experiments, Architecture 14 outperforms all the rest. The architecture's main characteristics are that it uses features extracted using the pretrained model as a board embedding. Additionally, the model uses two different dedicated user embeddings of size 25, one is used for inner product with the board embedding and one is concatenated to the rest of the input and passed through multiple fully connected layers. The model described in the Methodology section and that is later used in the extensive experiments is based on this architecture after conducting hyper parameter searching using TPE.

5.3 Blunder prediction results

A previous study [11], which had the same objective of predicting blunders in chess, has been able to achieve an accuracy score of 71.7% in their experiments, on a balanced dataset. We use their method as a point of comparison and apply it on our dataset for fair comparison.

With our finalized architecture, our model attained an AUC of 0.801 and an AUC-PR of 0.226 when evaluated on the complete test set as presented in Table 2.

Furthermore, we assessed the potential enhancement brought by incorporating user-specific information into the model by conducting an ablation study. This examination involved the evaluation of two supplementary models, both derived from the optimal blunder prediction model that utilizes the pretrained network for embedding. The results of the ablation study are shown in Table 3.

In the first model variant, the user's identifier, signifying the player about to execute the next move, was omitted, resulting in no user information incorporated in the

model. Consequently, no user embedding was employed in this configuration. Instead, the model was equipped with inputs including the board's embedding, the user's rating, the opponent's rating, and the move's ply (total move count of both players). Subsequent to processing the board representation through a fully connected neural layer housing 500 units and a ReLU activation function, the ensuing 500 features representing the board, and move ply were amalgamated into a singular vector. This composite vector was then channeled into another fully connected neural network, comprising of two layers housing 64 neurons with a ReLU activation function. The output of this stage, denoted as 'x,' proceeded to traverse through an additional fully connected neural network, comprising four layers, each encompassing 256 units and employing ReLU activation. Notably, the terminal layer of this network comprised a solitary unit, utilizing the Sigmoid activation function.

Conversely, the second model—akin to the initial variant—excluded the user's identifier. The next iteration of the model dispensed with the player's identifier, but incorporates the ELO rating regarding both the user and the opponent. Additionally, the last iteration is akin to the final model but includes the respective ratings of the player and the opponent. Through this rigorous analysis, we aimed to discern the extent to which personalized information influences the model's predictive capabilities, thereby enriching our understanding of the intricate interplay between user characteristics and gameplay outcomes.

Upon exclusion of the user's identifier as an input, the model experiences a reduction in performance metrics. The AUC either diminishes from 0.798 to 0.78 or from 0.8 to 0.79, accompanied by a corresponding drop in AUC-PR from 0.22 to 0.2 or from 0.22 to 0.21 respectively. Moreover, when both the user's and opponent's Elo ratings are removed from the model's input, a less substantial degradation in performance becomes evident. The AUC exhibits an increase from 0.78 to 0.79, and the AUC-PR follows suit, increasing from 0.2 to 0.21, compared to when just the user identified is excluded, demonstrating how including the Elo rating can degrade the performance as shown in Table 2. We hypothesize that the degradation in performance stems from the generalized nature of the Elo rating. As Elo measures only a player's aggregate strength, it obscures the critical playstyle and weakness variance present among equally-rated individuals. Because our model's dedicated embeddings are optimized to capture these precise, style-specific vulnerabilities, the inclusion of the coarse Elo metric introduces non-specific information that hinders predictive precision. The differences in performance between each pair of models for both the AUC and AUC-PR is statistically proven using Pairwise DeLong statistical test, the p-values are shown in Table 3. These findings collectively indicate that

Table 2 Evaluating the effectiveness of various user information incorporation strategies in the final blunder prediction model

Model name	AUC	AUC-PR
Model with Elo ratings and user identifier	0.798	0.221
Model with user identifier only	0.801	0.226
Model with Elo ratings only	0.78	0.2
Model without any user information	0.79	0.21
McIlroy 2020 approach	0.567	0.08

Table 3 Pairwise delong analysis for model performance comparison by AUC

Model name	Full model	Identifier only	Elo rating only	No user information
Full model	x	3.5e-10	1.28e-189	1.04e-18
Identifier only	3.5e-10	x	8.02e-204	1.58e-40
Elo rating only	1.28e-189	8.02e-204	x	1.2e-148
No user info	1.04e-18	1.58e-40	1.2e-148	x

the inclusion of the user's identifier significantly contributes to the blunder prediction model's efficacy, while including the player's rating might hurt the performance. This discernment underscores the nuanced interplay between user-specific attributes and gameplay outcomes, shedding light on the multifaceted dynamics that inform blunder prediction in the context of chess.

We conducted an assessment of [11]'s approach, this time employing the method on the same dataset subset as the one utilized for evaluating our method. Our evaluation protocol entailed an initial division of the training set into a 90/10 split, designating a segment for the validation dataset. We then proceeded to train the model on the modified training set, determining the optimal number of training steps required to attain peak performance while averting overfitting concerns. Subsequently, we trained the model using the entire original training set, employing the same number of training steps.

Despite our meticulous approach, the obtained AUC on the test set used to evaluate our method amounted to 0.567. This notably lower performance can be primarily attributed to the considerable disparity in dataset size. To elaborate, our training dataset encompasses slightly over 2.3 million chess moves, incorporating over 180k blunders. In contrast, [11] utilized a dataset comprising 182 million blunders and 272 million non-blunders, totaling over 454 million chess moves.

5.4 Blunder prediction by type

5.4.1 General blunder prediction model

For assessing the performance of our model on different types of blunders, we partitioned our test set into two subsets, each focusing on a specific blunder type. These subsets comprised moves categorized either as blunders of the respective type or as non-blunders, removing moves that are blunders of a different type. Subsequently, we evaluated the model's performance by computing the AUC on each of these new test sets. The results revealed that our model achieved an AUC of 0.79 for immediate blunders and 0.80 for non-immediate blunders.

5.4.2 Specific blunder types prediction models

Next, for each blunder type, we constructed new training and testing datasets, encompassing all moves from the original dataset and classifying a move as a blunder only if it pertained to the respective blunder type. Subsequently, we developed separate models using the same architecture as the full blunder prediction model to predict each relevant blunder type, resulting in two distinct models. Leveraging

these specialized models, we attained an AUC of 0.814 and an AUC-PR of 0.07 for immediate blunders, and an AUC of 0.80 and an AUC-PR of 0.19 for non-immediate blunders. The disparity in AUC-PR values stems from the fact that immediate blunders constitute merely 1.6% of the test set, whereas non-immediate blunders make up 6% of the test set. These specialized models achieve similar performance to the generic blunder prediction model when applied on datasets containing only non-blunders and blunders of the respective type. While also allowing us to differentiate between the blunder types, e.g., we could alert on a blunder only if it's an immediate blunder.

5.5 User analysis

We evaluated the model's performance across users with varying Elo ratings, to examine if the model's performance of predicting blunders is affected by the users' skill level. We categorized the users into five groups based on their elo ratings : 630-1284, 1285-1466, 1467-1632, 1633-1830, and 1831-2390. Where the different Elo rating groups were decided upon based on equal frequency binning. We conducted pairwise comparisons of the AUC and AUC-PR between each pair of groups using the Non-parametric Kruskal-Wallis rank sum test with significance threshold set at $p < 0.05$.

The mean AUC scores for the groups are 0.81, 0.8, 0.79, 0.786, and 0.78 respectively. Statistical analysis revealed that the only pairs of groups with similar performance are groups 3 and 4, as well as groups 4 and 5, as observed in Fig. 2.

In terms of AUC-PR, the values for the groups are as follows: 0.262, 0.253, 0.241, 0.23, and 0.228, respectively. Group 1 differs from groups 3-5, while group 2 differs from groups 4-5, as observed in Fig. 3.

The model exhibits diminished performance among player groups with elevated Elo ratings. This decline in performance may arise from the increased intricacy involved in modeling players at higher skill levels. Moreover, players at advanced levels tend to commit fewer blunders overall, resulting in a scarcity of blunder patterns for the model to effectively learn from. Crucially, as illustrated in Fig. 4, this

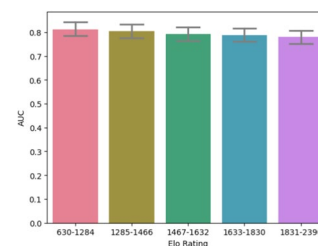


Fig. 2 AUC of the blunder prediction model on the test set by Elo rating

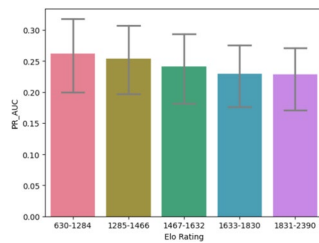


Fig. 3 AUC-PR of the blunder prediction model on the test set by Elo rating

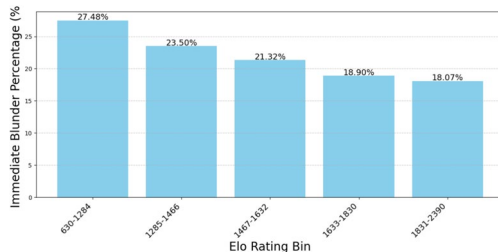


Fig. 4 Percentage of immediate blunders by Elo rating

scarcity is most pronounced for immediate blunders. As rating increases, players successfully filter out these simpler tactical oversights, shifting the distribution of remaining errors heavily toward non-immediate, strategic blunders. Since these long-term errors are subtler and less defined by distinct, concrete board patterns than immediate tactical slips, the prediction task becomes inherently more ambiguous and difficult in the high-Elo regime.

5.6 Blunder prediction by game phase

We conducted an analysis of the model's performance on boards from different game phases: opening, middle-game, and end-game, the results are shown in Table 4. The criteria for dividing the game into opening, middle and end game phases is commonly defined based on Lichess.org [10]. The opening phase technically lasts as long as the current board position matches a known game in the opening database, the endgame is defined as when the total number of pieces on the board (excluding kings and pawns) drops to six or fewer, and the middle is between the opening and the end game. Using Lichess' codebase [10] to identify the phase of the game for each board position, we evaluated the model's AUC and AUC-PR across these phases. The results for the opening phase show an AUC of 0.8 and an AUC-PR of 0.213, based on a sample of 114,655 moves, including 7,924

Table 4 Model performance by game phase

Game phase	AUC	AUC-PR	Sample size	Blunder count
Opening	0.8	0.213	114655	7924
Middle	0.766	0.248	123551	12811
End-game	0.835	0.178	86768	4265

blunders. For the middle game, the AUC was 0.766 and the AUC-PR was 0.248, with a sample of 123,551 moves and 12,811 blunders. The model's performance during the end game yielded an AUC of 0.835 and an AUC-PR of 0.178, with a sample size of 86,768 moves and 4,265 blunders. While the model achieves its highest discriminative performance (AUC of 0.835) in the endgame, the AUC-PR of 0.178 is notably lower than in the opening and middle game phases. This is a result of the decreased percentage of blunders in the endgame test set leading to a lower baseline for the AUC-PR metric.

The variation in AUC-PR scores between the game phases can be attributed to the differing number of blunders (positives) in each phase. The subset of boards that belong to endgame, for example, has a significantly lower number of positives compared to the middle-game, which naturally affects the AUC-PR due to the imbalanced class distribution. Overall, we demonstrate an improvement of 0.129 to 0.144 compared to the AUC-PR of the null model.

Given the varying positive sample rates across the individual game phase datasets, the AUC is utilized as the most appropriate metric for performance comparison, as it remains independent of the sample ratio. The model's performance exhibits a clear correlation with the structural and strategic demands of each game phase, as evidenced by the distinct AUC scores: 0.800 for the opening, 0.766 for the middle game, and 0.835 for the endgame. The highest AUC (0.835) is achieved in the endgame, where the reduction in material leads to simpler positions and the model can reliably focus on essential technical relationships. Conversely, the lowest performance (AUC 0.766) is observed in the middle game, which features the maximum complexity and positional novelty, challenging the model's ability to generalize intricate tactical and strategic patterns. The opening phase, despite the high material count, maintains a strong AUC of 0.800. This performance is attributed to the structural commonality and high frequency of opening positions, which in turn ensures reliable feature extraction by the pre-trained CNN and our model on these established schemes.

6 Conclusion

6.1 Summary of results

This research successfully developed a novel, personalized blunder prediction model for chess. We demonstrated that a single, unified collaborative model can achieve state-of-the-art performance (0.801 AUC). This collaborative architecture proved highly effective, significantly outperforming the baseline method [11] on our dataset showing

its effectiveness on smaller datasets, challenging the “one-model-per-player” paradigm. Our most significant finding is that a dynamically learned latent ‘blunder profile’ is a more powerful and nuanced predictor of error than a player’s explicit Elo rating. Furthermore, we established a new formulation for this task by successfully training the model to differentiate between immediate (tactical) and non-immediate (strategic) blunders, paving the way for more targeted training tools.

6.2 Broader implications and contributions

This work has several broader implications for the fields of user modeling and computational chess analysis. Firstly, it enables personalized modeling of player skills, weaknesses, and blunder likelihoods through the utilization of user embeddings. This personalized approach allows a deeper understanding of individual player tendencies and more accurate predictions. Additionally, our model uses collaborative filtering, allowing knowledge sharing between players. This contrasts with previous approaches that necessitated a unique model for each player, resulting in longer training times and reduced efficiency. The collaborative architecture is highly efficient because the shared weights of the DeepFM and CNN components are universally optimized to model blunder patterns across the entire player population, meaning that for a new user, the system only needs to determine their latent embedding vector, which can be efficiently initialized based on existing users and new players can eventually align with the proven, high-performance profiles of previously trained players. Moreover, our method employs end-to-end learning, which allows for the direct extraction of features from raw board positions, streamlining the modeling process and enhancing overall effectiveness.

Beyond its theoretical contributions to personalized deep learning, this research holds significant practical value for the development of next-generation intelligent tutoring systems. By accurately modeling individual blunder tendencies rather than just general move quality, chess platforms can transition from generic engine analysis to highly personalized coaching. This framework enables the creation of adaptive training modules that automatically identify a player’s specific cognitive blind spots—such as susceptibility to specific tactical motifs or time-pressure errors and generate targeted drills to mitigate them. Ultimately, this approach democratizes access to high-level, style-aware instruction, making sophisticated, data-driven pedagogy accessible to the broader chess community. In addition, we can provide separate results for immediate and non-immediate blunders allowing for further personalized tutoring. We have shown

that our model is effective for users with varying skill levels and for varying game phases.

6.3 Future research

Future research can build upon this work in multiple directions: Several future research directions stem from our work. Enhancing model interpretability through attention layers or gradient-based attribution methods could provide insights into decision-making processes. Integrating game phase and strategy information into board representations may improve predictive capabilities. Exploring alternative architectures like graph neural networks or a single multi-task learning model with a multi-class output for predicting blunder types simultaneously, offers potential for enhancing model performance. Leveraging reinforcement learning alongside human data could advance game proficiency. Extending representation learning to other strategic board games widens the methodology’s applicability. Enhancing player characteristic prediction with larger datasets and more modalities could refine player behavior understanding. Investigating cheating detection through anomalous gameplay analysis presents a promising avenue. In addition, we believe that adding explainability to the blunder might help the learning process and might increase trust in the system. Integrating chess board embedding methodologies with blunder prediction into chess puzzle recommendation systems may enrich player learning. Finally, exploring correlations between chess moves and player characteristics like age, fatigue and location offers insights into player demographics and preferences.

In conclusion, this paper introduces impactful techniques for personalized modeling, blunder prediction, and player characterization in chess using deep learning. While focused on chess, the approaches may generalize to other game domains (e.g., Go). The representations and prediction models provide a strong foundation for future research to enhance understandings of gameplay strategy and cognition.

Funding Open access funding provided by Ben-Gurion University.

Open Access This article is licensed under a Creative Commons Attribution 4.0 International License, which permits use, sharing, adaptation, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if changes were made. The images or other third party material in this article are included in the article’s Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article’s Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit <http://creativecommons.org/licenses/by/4.0/>.

References

1. Albers PCH, Vries HD (2001) Elo-rating as a tool in the sequential estimation of dominance strengths. *Animal behaviour*, pp 489–495
2. Chen C, Kim S, Bui H, Rossi R, Koh E, Kveton B, Bunescu R (2018) Predictive analysis by leveraging temporal user behavior and user embeddings. In: *Proceedings of the 27th ACM international conference on information and knowledge management*, pp 2175–2182
3. Czech J (2019) Deep reinforcement learning for crazyhouse. M.sc, TU Darmstadt
4. Czech J, Willig M, Beyer A, Kersting K, Fürnkranz J (2020) Learning to play the chess variant crazyhouse above world champion level with deep neural networks and human data. *Front Artif Intell* 3:24
5. Fischer G (2001) User modeling in human-computer interaction. *User Model User-Adap Inter* 11(1):65–86
6. Guo H, Tang R, Ye Y, Li Z, He X (2017) Deepfm: a factorization-machine based neural network for ctr prediction. [arXiv:1703.04247](https://arxiv.org/abs/1703.04247)
7. Huang K-H, Deng Y-S, Chuang M-C (2012) Static and dynamic user portraits. *Adv Human-Comput Interact* 2012
8. Li S, Zhao H (2020) A survey on representation learning for user modeling. In: *IJCAI*, pp 4997–5003
9. Lian R, Huang C-W, Tang Y, Gu Q, Ma C, Guo C (2022) Incremental user embedding modeling for personalized text classification. In: *ICASSP 2022–2022 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*. IEEE, pp 7832–7836
10. Lichess (2024) Scala chess. https://github.com/lichess-org/scalac_hess
11. McIlroy-Young R, Sen S, Kleinberg J, Anderson A (2020) Aligning superhuman ai with human behavior: Chess as a model system. In: *Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, pp 1677–1687
12. McIlroy-Young R, Wang R, Sen S, Kleinberg J, Anderson A (2022) Learning models of individual behavior in chess. In: *Proceedings of the 28th ACM SIGKDD conference on knowledge discovery and data mining*, pp 1253–1263
13. Pan S, Ding T (2019) Social media-based user embedding: A literature review. [arXiv:1907.00725](https://arxiv.org/abs/1907.00725)
14. Rendle S (2010) Factorization machines. In: *2010 IEEE International conference on data mining*. IEEE, pp 995–1000
15. Saariluoma P (1992) Error in chess: The apperception-restructuring view. *Psychol Res* 54:17–26
16. Schnabel T, Wan M, Yang L (2022) Situating recommender systems in practice: Towards inductive learning and incremental updates. [arXiv:2211.06365](https://arxiv.org/abs/2211.06365)
17. Silver D, Hubert T, Schrittwieser J, Antonoglou I, Lai M, Guez A, Lanctot M, Sifre L, Kumaran D, Graepel T et al (2018) A general reinforcement learning algorithm that masters chess, shogi, and go through self-play. *Science* 362(6419):1140–1144
18. Singla K, Abrol S, Park S (2019) User embeddings based on mobile app behavior data. In: *International conference on smart homes and health telematics*. Springer, pp 183–189
19. Wang P, Liu K, Jiang L, Li X, Fu Y (2020) Incremental mobile user profiling: Reinforcement learning with spatial knowledge graph for modeling event streams. In: *Proceedings of the 26th ACM SIGKDD international conference on knowledge discovery & data mining*, pp 853–861
20. Webb GI, Pazzani MJ, Billsus D (2001) Machine learning for user modeling. *User Model User-Adap Inter* 11(1):19–29
21. Wu X, Lin W, Wang Z, Rastorgueva E (2020) Author2vec: A framework for generating user embedding. [arXiv:2003.11627](https://arxiv.org/abs/2003.11627)
22. Xu Y, Zhang Y, Guo W, Guo H, Tang R, Coates M (2020) Graphsail: Graph structure aware incremental learning for recommender systems. In: *Proceedings of the 29th ACM international conference on information & knowledge management*, pp 2861–2868
23. Zhang F, Yuan NJ, Lian D, Xie X, Ma W-Y (2016) Collaborative knowledge base embedding for recommender systems. In: *Proceedings of the 22nd ACM SIGKDD international conference on knowledge discovery and data mining*, pp 353–362
24. Zhang S, Yao L, Sun A, Tay Y (2019) Deep learning based recommender system: A survey and new perspectives. *ACM Comput Surv (CSUR)* 52(1):1–38

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.